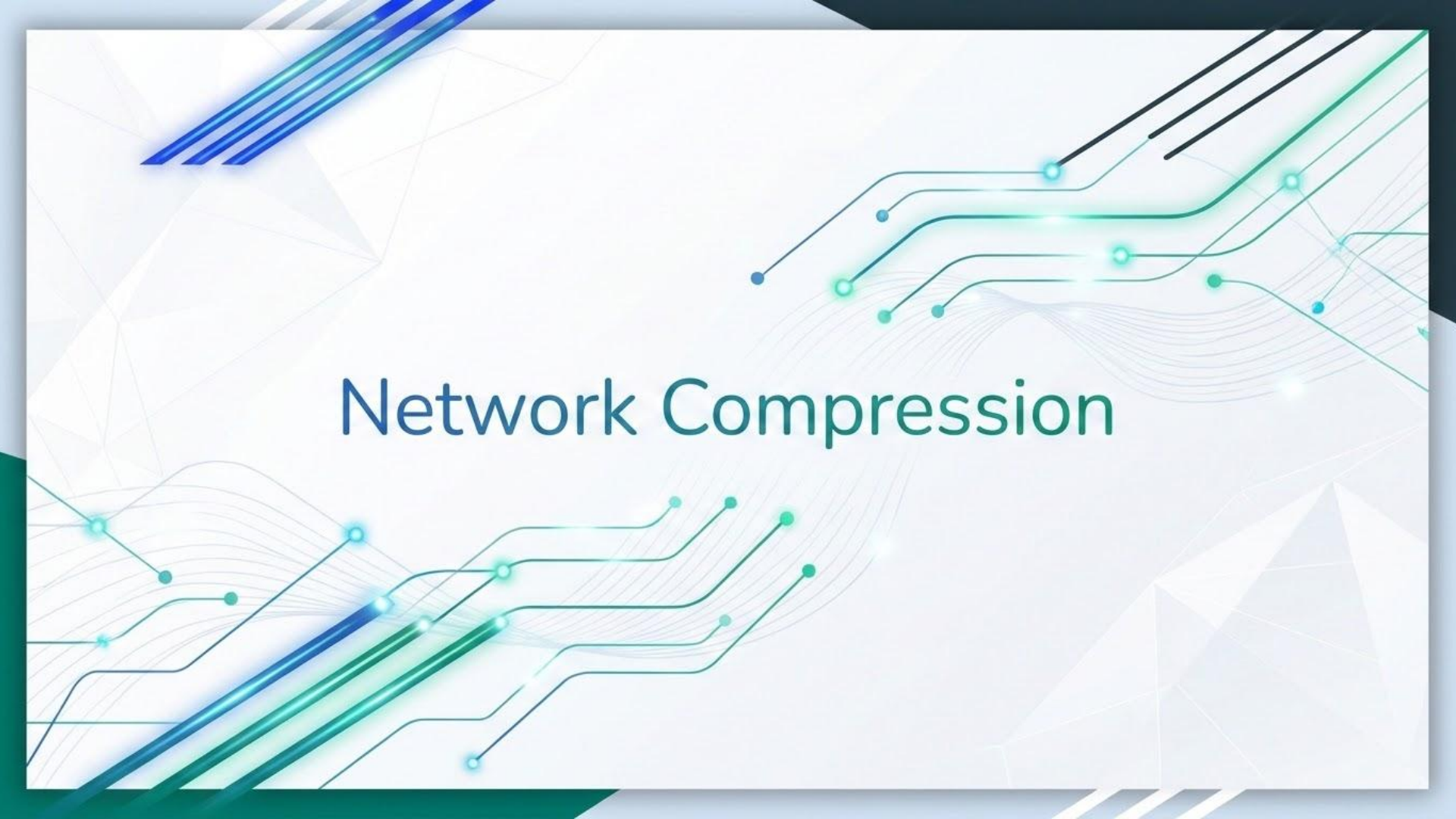
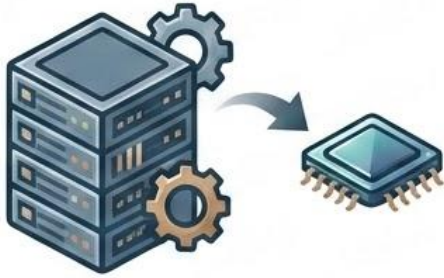


The background is a light blue and white abstract design. It features several glowing blue and green lines that resemble circuit traces or data paths, some with small circular nodes. There are also faint, light blue geometric shapes, including triangles and polygons, scattered across the background. The overall aesthetic is clean, modern, and technological.

Network Compression

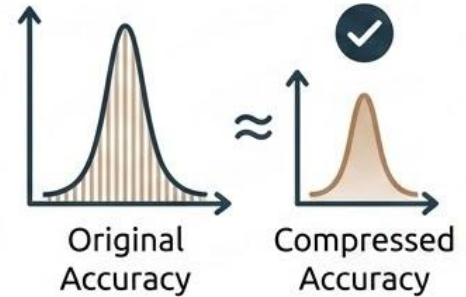
Why do we want to compress the network?



- Reduce the storage and computation requirements.



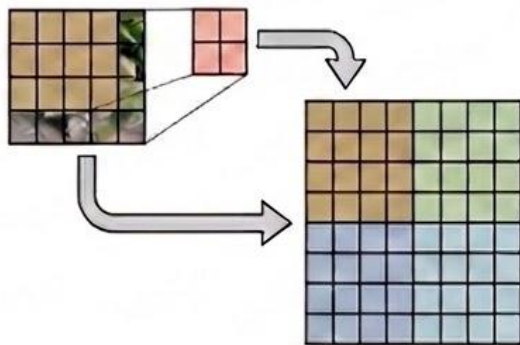
- We want to deploy the DNN to the IOT devices.



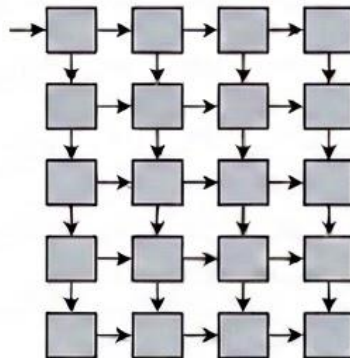
- We want to keep the accuracy as closed as possible.

How to accelerate the model inference

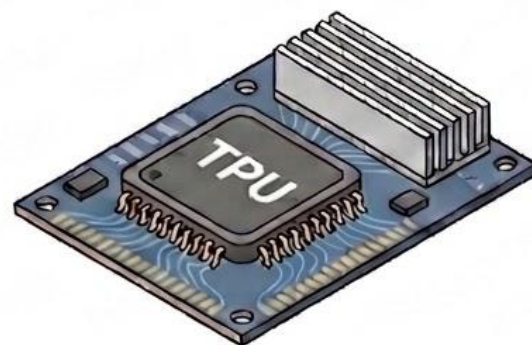
im2col



systolic array



Google TPU

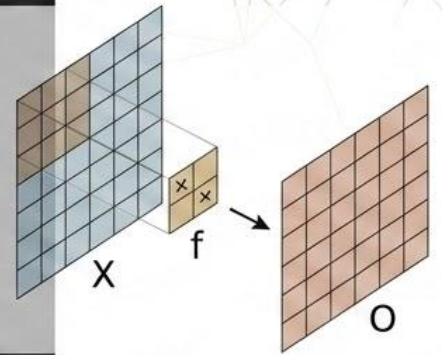


How do we implement matrix multiplication?

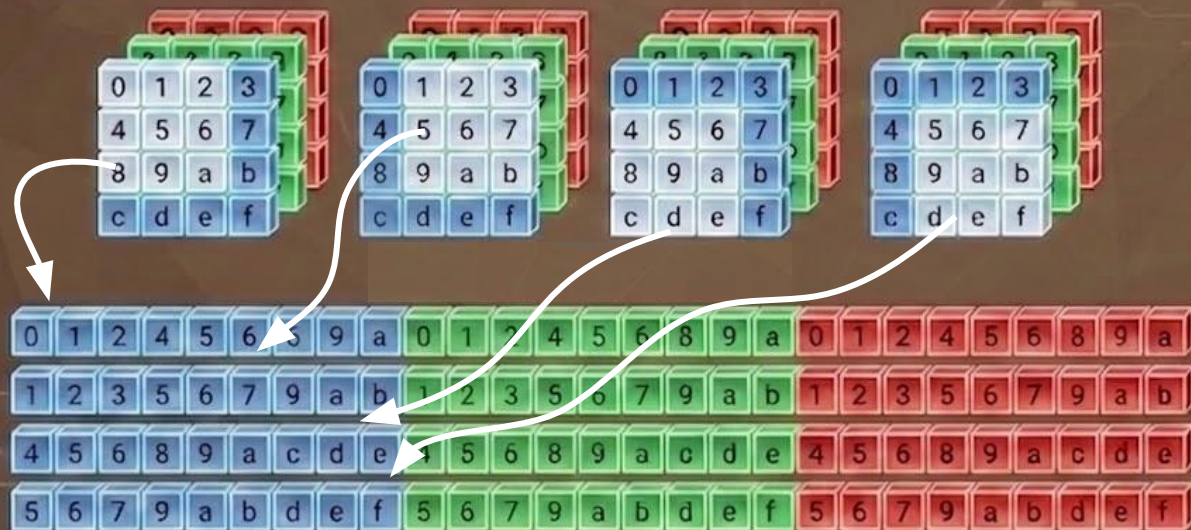
- Data is naturally represented as vectors
- Models need to transform vectors $\rightarrow$ new vectors
 - $Y = X * W$
- most natural way to transform vectorized data
- most efficient operation on modern hardware
- foundation of scalable gradient learning
- Universal Building Block
 - Linear
 - Convolution
 - Transformer

How do we implement CNN?

```
1  for x in range(w):  
2      for y in range(h):  
3          v = 0;  
4          for fx in range(fW):  
5              for fy in range(fH):  
6                  v += X[y+fy,x+fx] * f[y,x]  
7          O[y,x] = v
```



im2col Single output channel



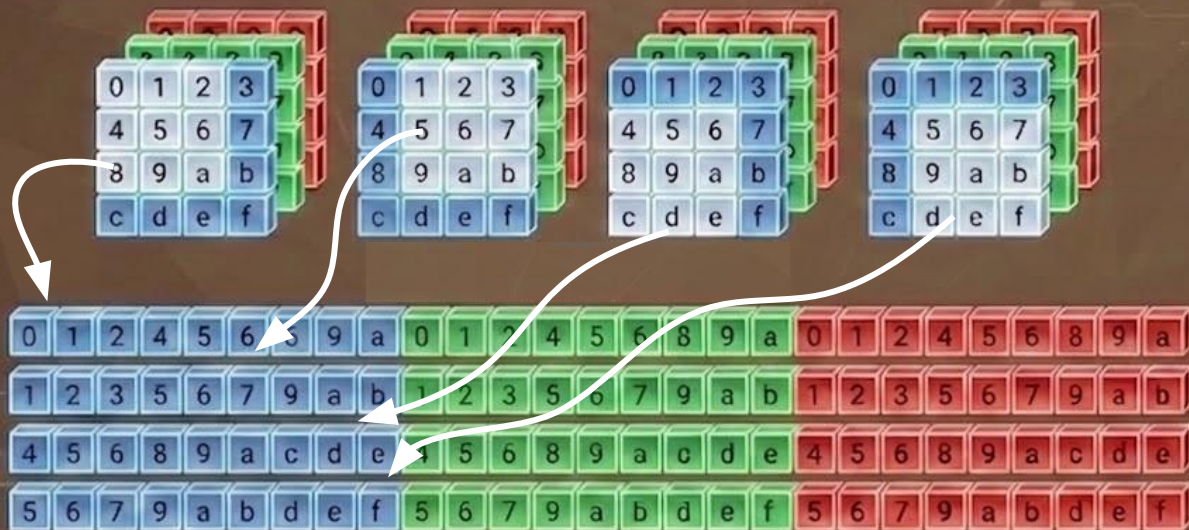
$4 \times (3 \times 3 \times 3) \implies 4 \times 27$

$3 \times 3 \times 3 \implies 27$

27×1



im2col multiple output channel



$4 \times (3 \times 3 \times 3) \Rightarrow 4 \times 27$

0	0	0
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7
8	8	8
0	0	0
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7
8	8	8
0	0	0
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7
8	8	8

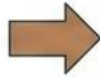
$3 \times 3 \times 3 \Rightarrow 27$

27×3

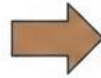
How to fully utilize the hardware?



**SMALL MATRIX
MULTIPLICATION**



**GIANT MATRIX
MULTIPLICATION**



According to the capability of the hardware



The supported shape of matrix multiplication



The number of multiplication units



The memory architecture



You need to learn the Linear algebra.

Systolic array

- The neural network is the combination of the following operations
 - Convolution, Relu, polling, filly-connected layer(dense), softmax
- We can implement those operations in a systolic array.
- It is a collection of MAC organized as an matrix.
- It is very efficient for hugh matrix multiplication.
- It can not handle irregular/sparse networks.
- Most current commercial and research project use it to implement NN in hardware
 - NVIDIA Tensor Core、Google TPU、Eyeriss I/II、ShiDianNao系列、Chain-NN、Cambricon-x

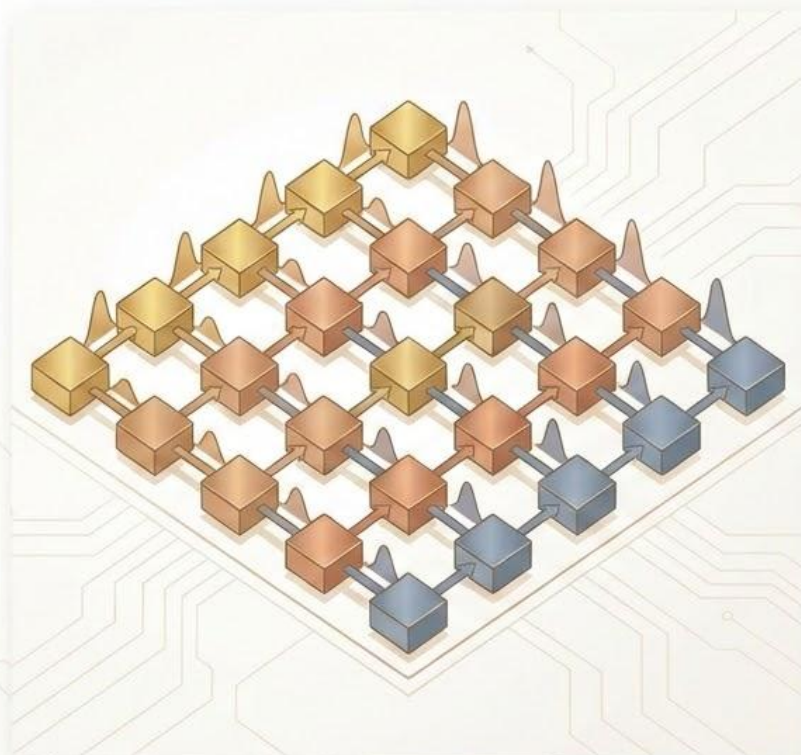
Systolic Array

<https://arxiv.org/pdf/1807.07928.pdf>

<https://kknews.cc/zh-tw/tech/qj8429.html>

<https://arxiv.org/pdf/1703.01457.pdf>

<http://csit.org/mediawiki/images/f/f1/Cambricon-X.pdf>



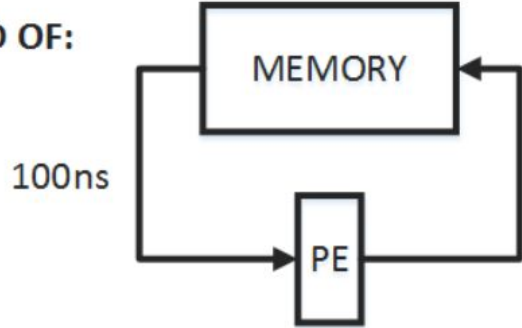
- The neural network is the combination of operations: Convolution, Relu, Pooling, Fully-connected layer (dense), Softmax.
 - Implemented efficiently in a systolic array.
 - A collection of MACs organized as a matrix.
 - Very efficient for huge matrix multiplication.
 - Cannot handle irregular/sparse networks.
- ### Hardware Implementations
- NVIDIA Tensor Core, Google TPU, Eyeriss I/II, ShiDianNao系列, Chain-NN, Cambricon-x

Systolic array

- The neural network is the combination of the following operations
 - Convolution, Relu, polling, filly-connected layer(dense), softmax
- We can implement those operations in a systolic array.
- It is a collection of MAC organized as an matrix.
- It is very efficient for hugh matrix multiplication.
- It can not handle irregular/sparse networks.
- Most current commercial and research project use it to implement NN in hardware
 - NVIDIA Tensor Core、Google TPU、Eyeriss I/II、ShiDianNao系列、Chain-NN、Cambricon-x

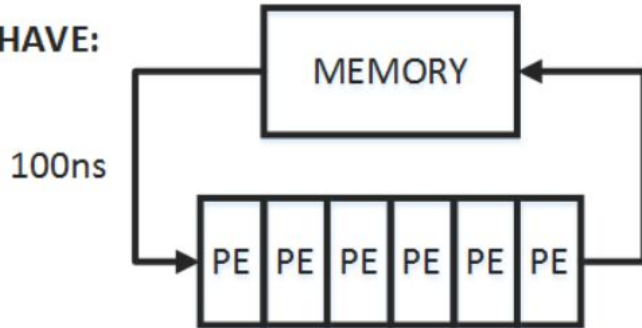
systolic array

INSTEAD OF:



5 MILLION
OPERATIONS
PER SECOND
AT MOST

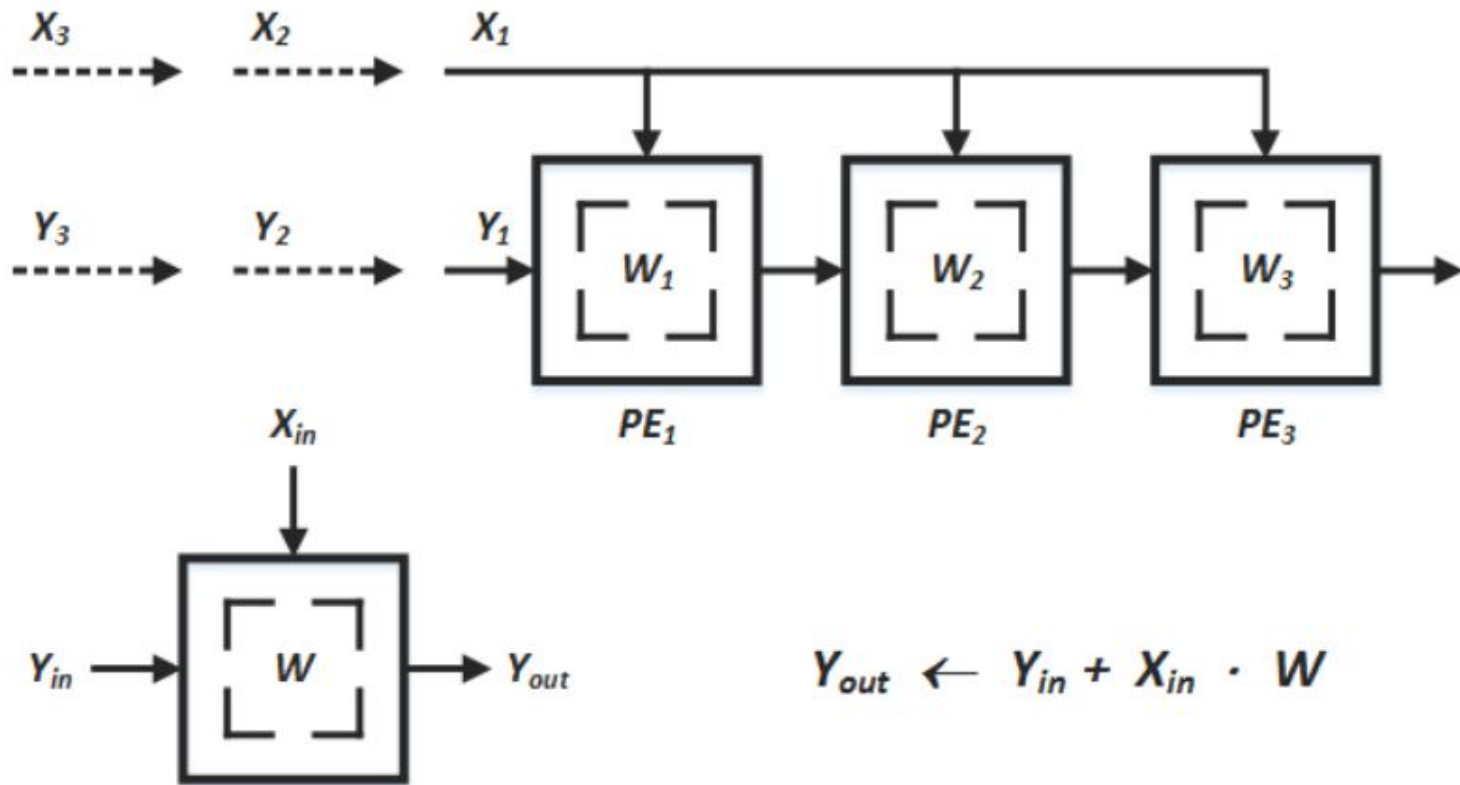
WE HAVE:

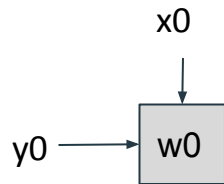


30 MOPS
POSSIBLE

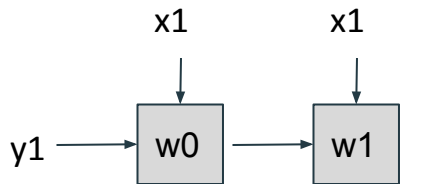
THE SYSTOLIC ARRAY

systolic array: 1D CNN





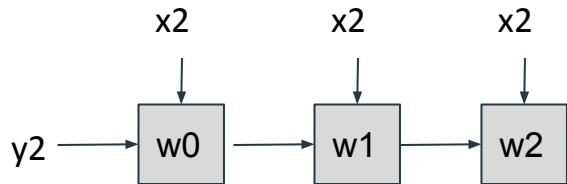
$$y_0 + x_0 * w_0$$



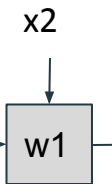
$$y_1 + x_1 * w_0$$



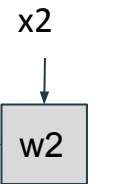
$$y_0 + x_0 * w_0 + x_1 * w_1$$



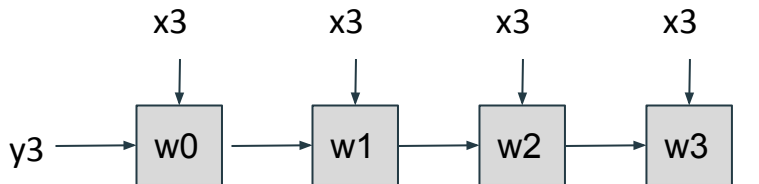
$$y_2 + x_2 * w_0$$



$$y_1 + x_1 * w_0 + x_2 * w_1$$



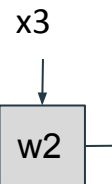
$$y_0 + x_0 * w_0 + x_1 * w_1 + x_2 * w_2$$



$$y_3 + x_3 * w_0$$



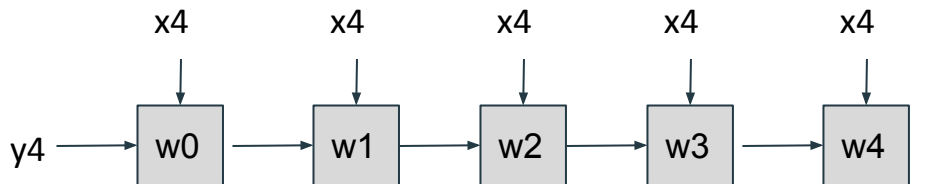
$$y_2 + x_2 * w_0 + x_3 * w_1$$



$$y_1 + x_1 * w_0 + x_2 * w_1 + x_3 * w_2$$



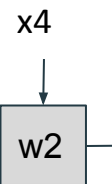
$$y_0 + x_0 * w_0 + x_1 * w_1 + x_2 * w_2 + x_3 * w_3$$



$$y_4 + x_4 * w_0$$



$$y_3 + x_3 * w_0 + x_4 * w_1$$



$$y_2 + x_2 * w_0 + x_3 * w_1 + x_4 * w_2$$



$$y_1 + x_1 * w_0 + x_2 * w_1 + x_3 * w_2 + x_4 * w_3$$



$$y_0 + x_0 * w_0 + x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + x_4 * w_4$$

Shift data to the right

Fetch data from the left

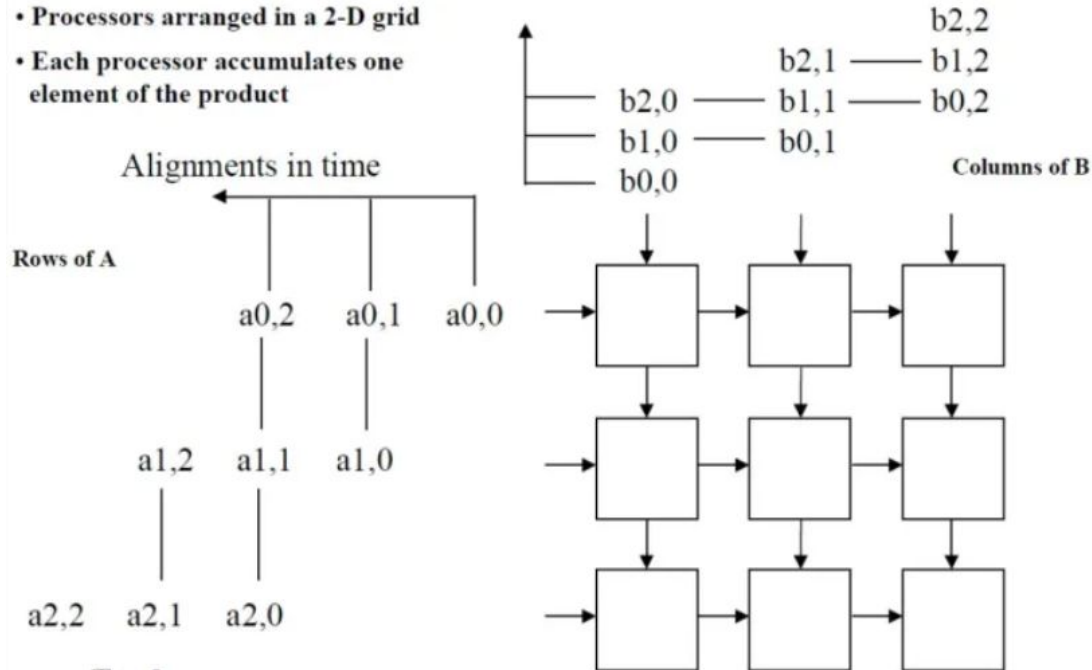
Performance MAC out = in + top_in * w

systolic array: GEMM(General Matrix Multiplication)

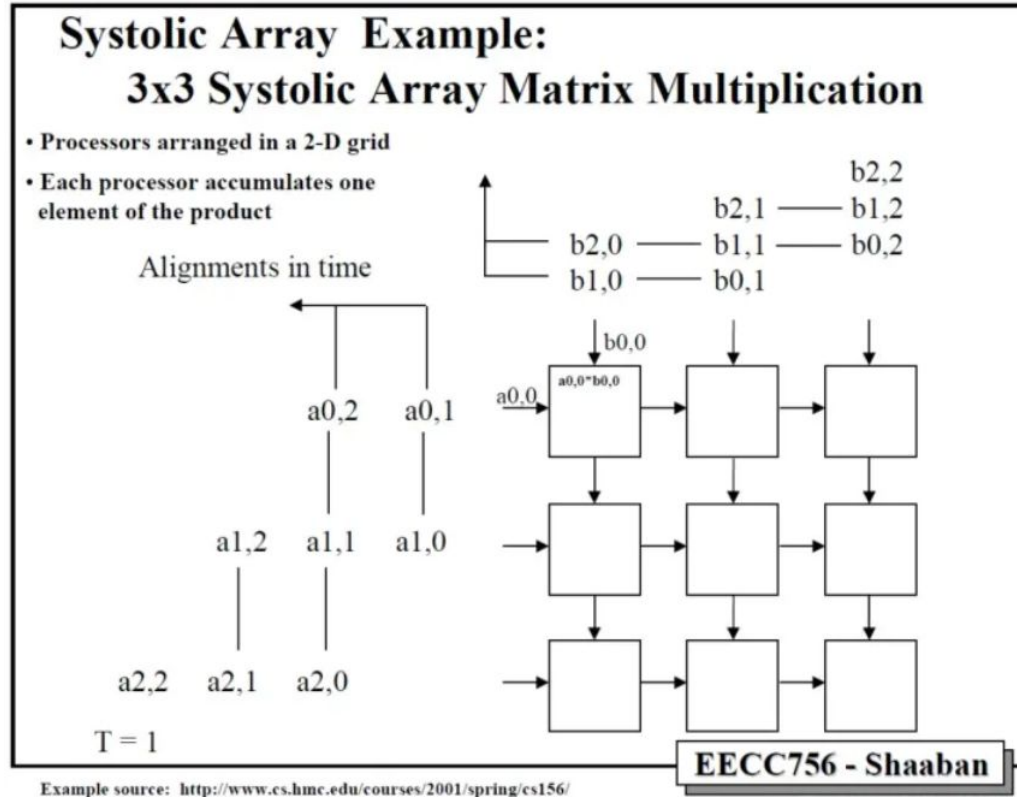
Systolic Array Example:

3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

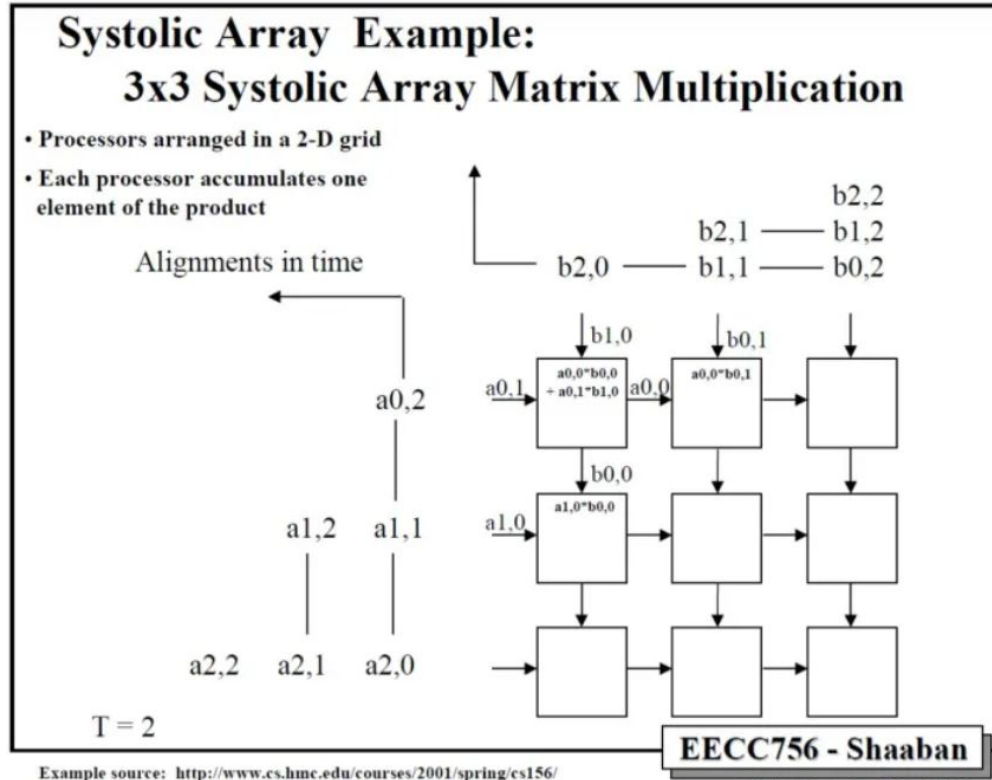


systolic array: GEMM(General Matrix Multiplication)

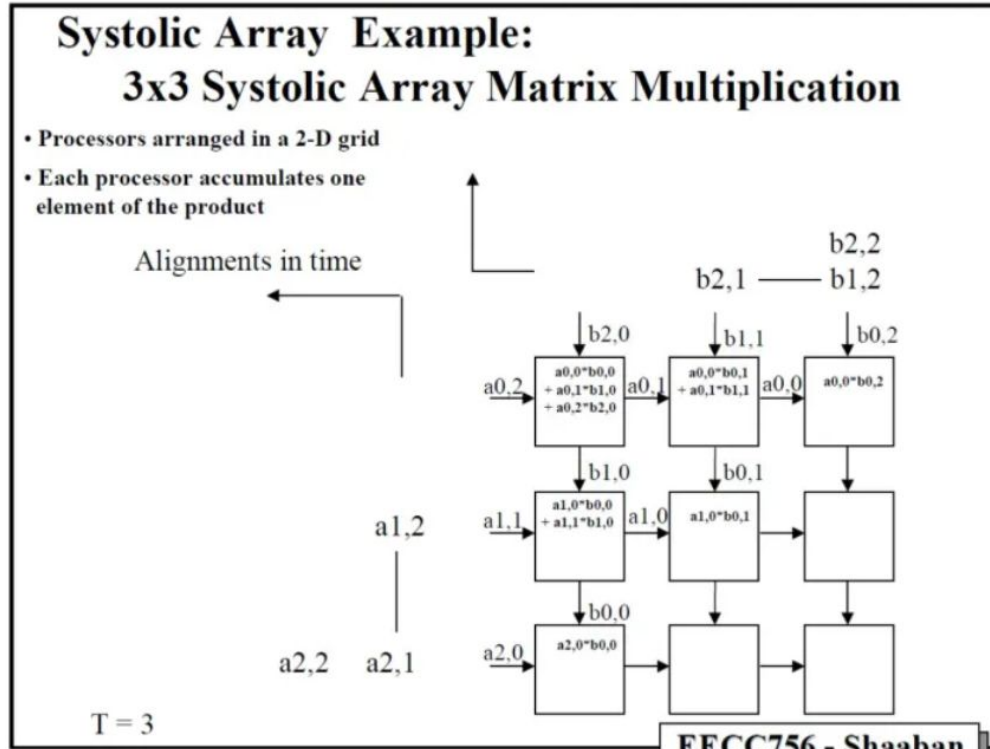


Example source: <http://www.cs.hmc.edu/courses/2001/spring/cs156/>

systolic array: GEMM(General Matrix Multiplication)



systolic array: GEMM(General Matrix Multiplication)



Example source: <http://www.cs.hmc.edu/courses/2001/spring/cs156/>

EECC756 - Shaaban

#5 lec #1 Spring 2003 3-11-2003

systolic array: GEMM(General Matrix Multiplication)

Systolic Array Example:

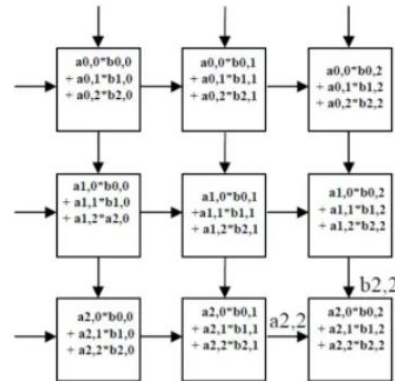
3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

Alignments in time

Done

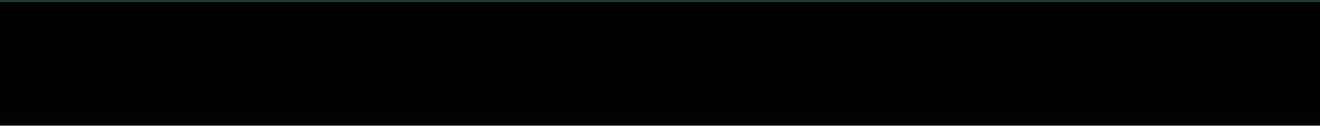
$T = 7$



EECC756 - Shaaban

Example source: <http://www.cs.hmc.edu/courses/2001/spring/cs156/>

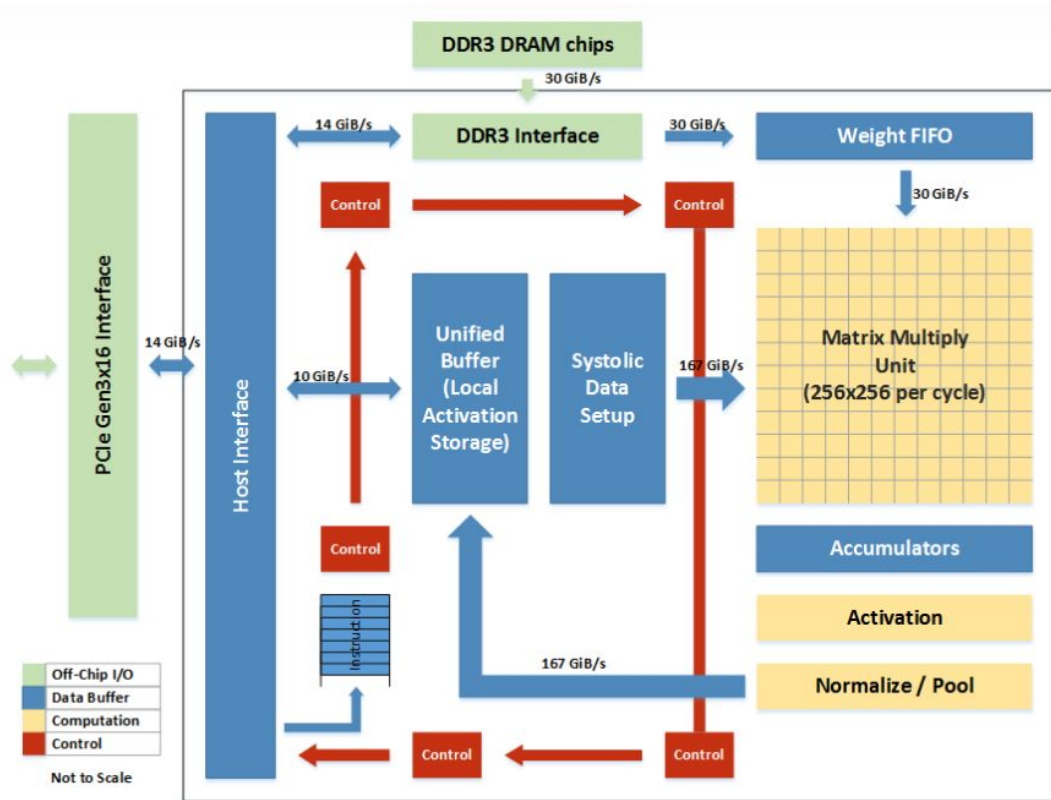
#9 lec #1 Spring 2003 3-11-2003



	0+ a14*b41			
1				

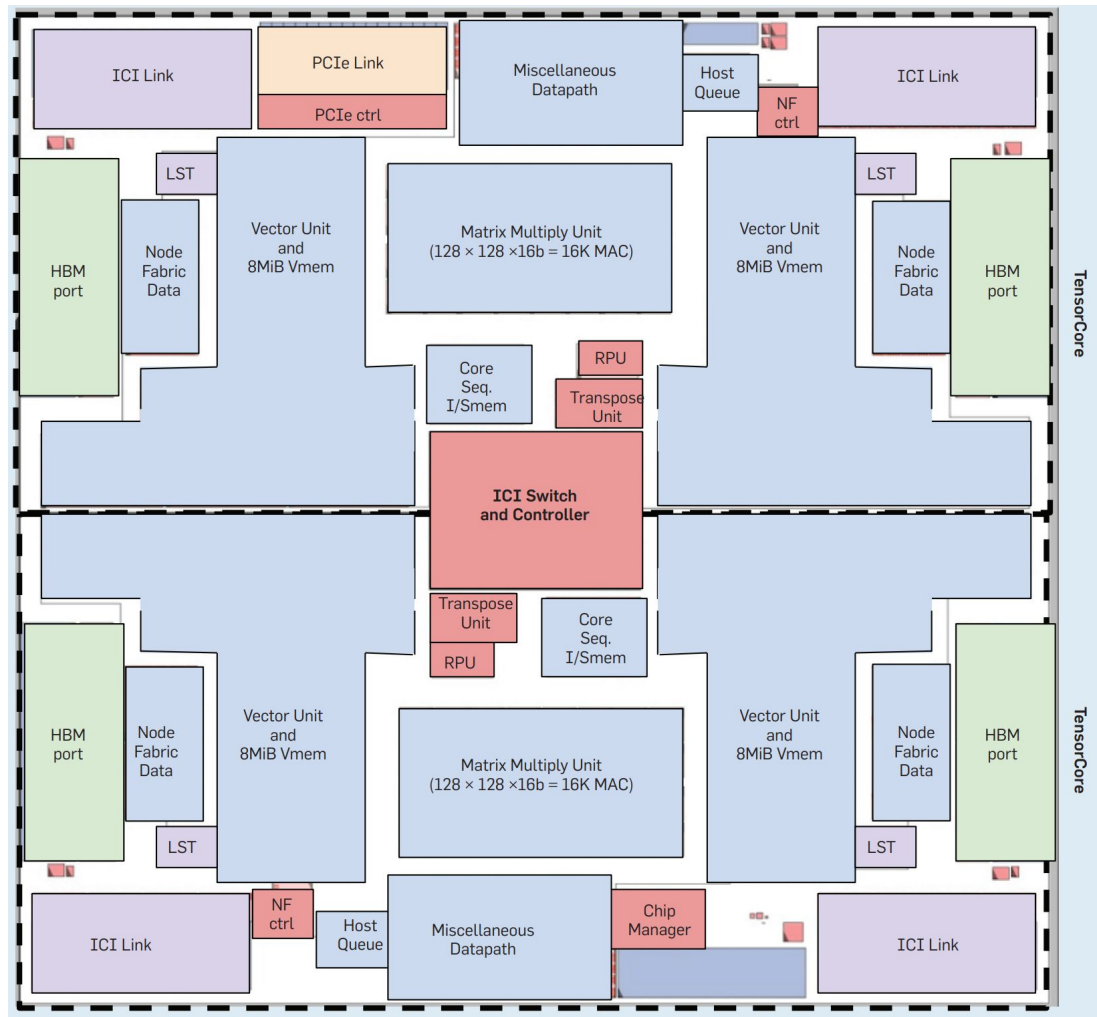


Google TPU V1



TPU v2/V3

- 256-1024 cores per system
 - connected in 2D structure

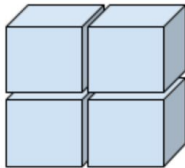


TPU v4

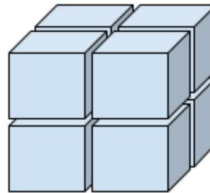
- 4096 cores per system
 - connected in 2D/3D structure

TPU v4 Common Topologies

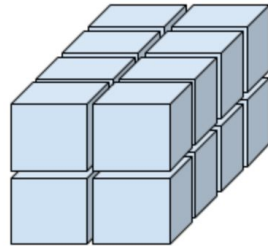
2x2x1
4 chips, 8 cores



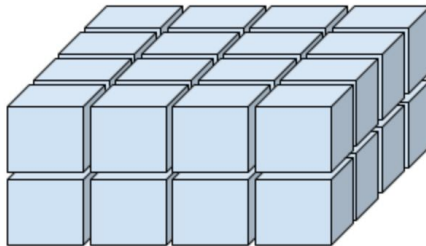
2x2x2
8 chips, 16 cores



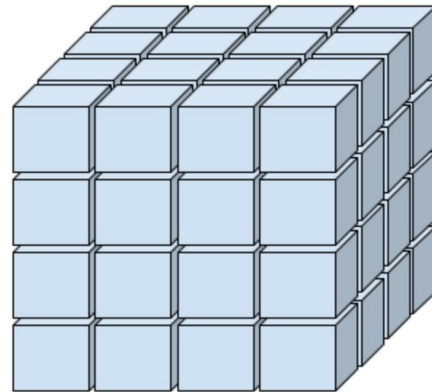
2x2x4
16 chips, 32 cores



2x4x4
32 chips, 64 cores

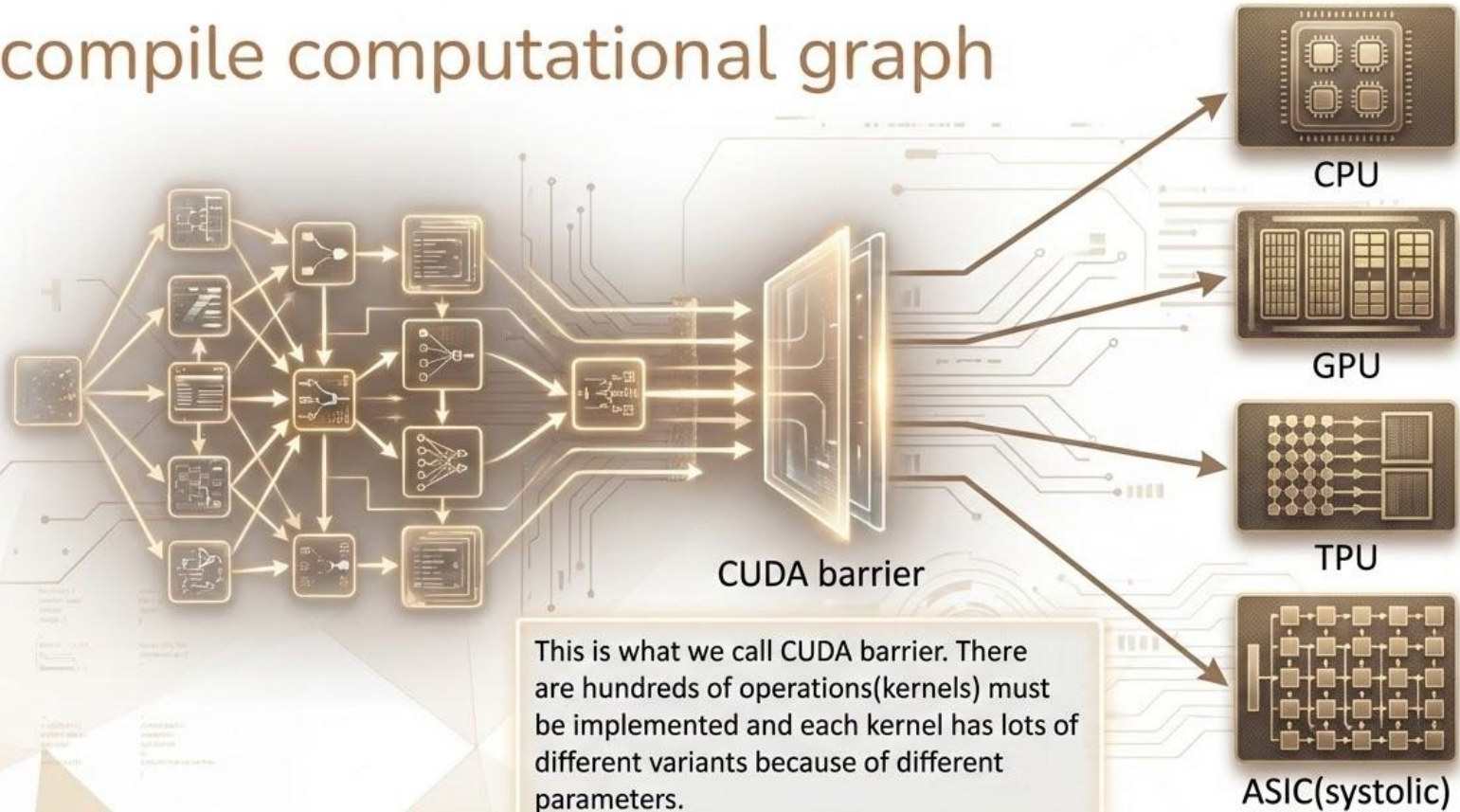


4x4x4
64 chips, 128 cores



Specification	v5p	v6e (Trillium)	TPU7x (Ironwood)
Number of chips per pod	8960	256	9216
Peak compute per chip (BF16) (TFLOPs)	459	918	2307
Peak compute per chip (FP8) (TFLOPs)	459	918	4614
HBM capacity per chip (GiB)	95	32	192
HBM bandwidth per chip (GiBps)	2575	1638	7380
Number of vCPUs (4-chip VM)	208	180	224
RAM (GB) (4-chip VM)	448	720	960
Number of TensorCores per chip	2	1	2
Number of SparseCores per chip	4	2	4
Bidirectional inter-chip interconnect (ICI) bandwidth per chip (GBps)	1200	800	1200
Data center network (DCN) bandwidth per chip (Gbps)	50	100	100

compile computational graph



How to compress the network

- Network pruning
- Parameter Quantization
- Low-Rank Decomposition
- [knowledge distillation](#)
- [Light-Weighted Model Design](#)

<https://arxiv.org/pdf/2010.03954.pdf>

<https://www.mdpi.com/2073-431X/12/3/60>

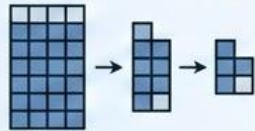
How to compress the network

- Network pruning
- Parameter Quantization
- Low-Rank Decomposition
- [knowledge distillation](#)
- [Light-Weighted Model Design](#)

How to compress the network

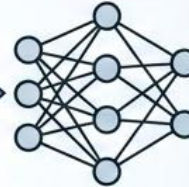
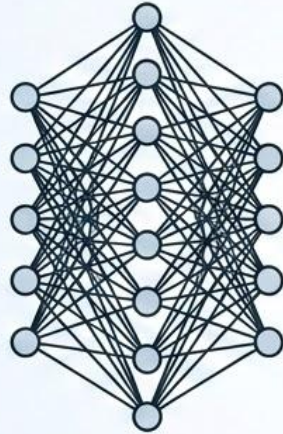


Network pruning

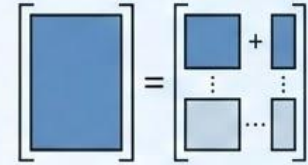


32-bit 32-bit 8-bit

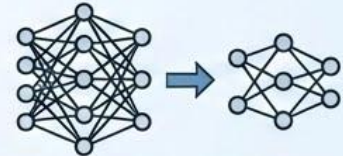
Parameter Quantization



Light-Weighted Model Design



Low-Rank Decomposition



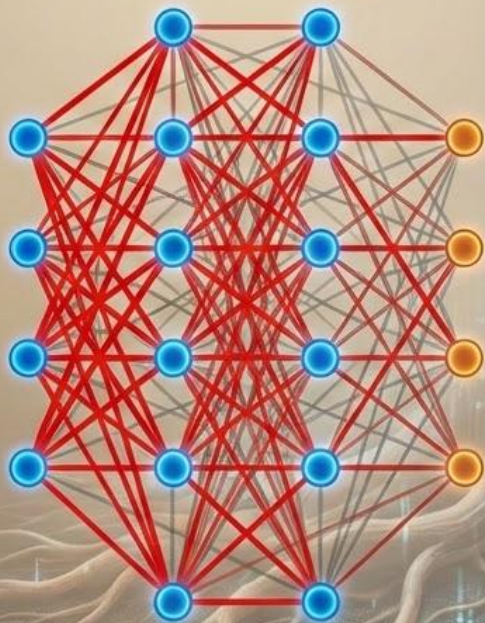
teacher

student

knowledge distillation

Connection pruning

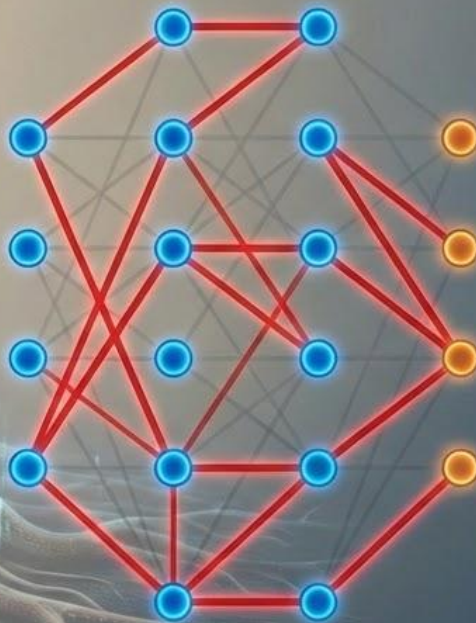
Before Pruning



Prune links
with small weight



After Pruning



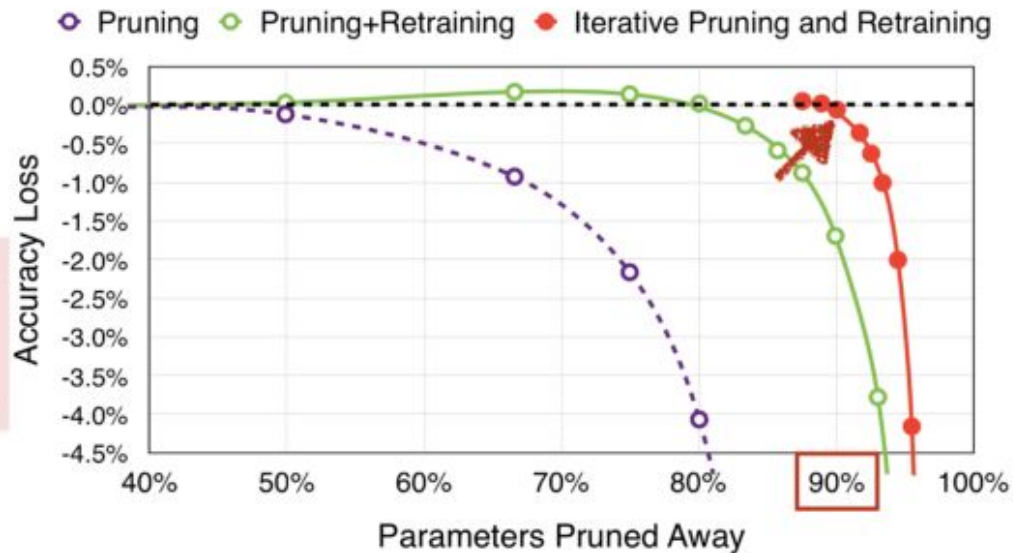
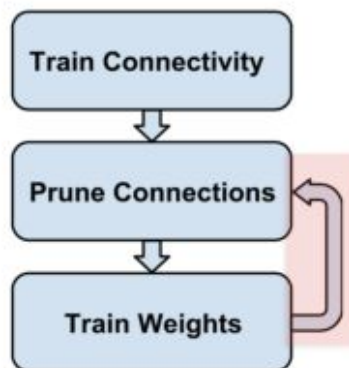
- The connection become irregular. 

- It's difficult to implement an efficient library to accelerate the computing. 

How to select weights to pruning

- Select the almost-zero weights $Y = \sum X_i \cdot W_i$
- How to force the weight to be zero $L = \sum W_i^2$
 - This is actually the weight decay

Iterative Pruning



Pruning procedure

- L1 or L2 regulation
 - L1 is better than L2 if no retrain
 - L2 + retrain is better in performance.
- Iterative retrain provide best performance.
- Train from scratch provide even better performance.
 - Pretrain is not required.

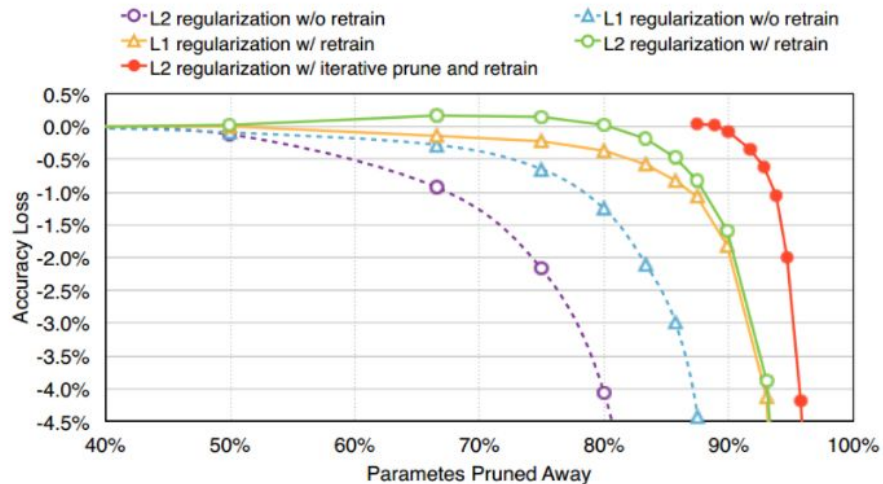


Figure 5: Trade-off curve for parameter reduction and loss in top-5 accuracy. L1 regularization performs better than L2 at learning the connections without retraining, while L2 regularization performs better than L1 at retraining. Iterative pruning gives the best result.

Will it speedup after pruning?

Table 4: Sparsity and speedup of *AlexNet* on ILSVRC 2012

#	Method	Top1 err.	Statistics	conv1	conv2	conv3	conv4	conv5
1	ℓ_1	44.67%	sparsity	67.6%	92.4%	97.2%	96.6%	94.3%
			CPU $\times$	0.80	2.91	4.84	3.83	2.76
			GPU $\times$	0.25	0.52	1.38	1.04	1.36
2	SSL	44.66%	column sparsity	0.0%	63.2%	76.9%	84.7%	80.7%
			row sparsity	9.4%	12.9%	40.6%	46.9%	0.0%
			CPU $\times$	1.05	3.37	6.27	9.73	4.93
			GPU $\times$	1.00	2.37	4.94	4.03	3.05
3	pruning[7]	42.80%	sparsity	16.0%	62.0%	65.0%	63.0%	63.0%
4	ℓ_1	42.51%	sparsity	14.7%	76.2%	85.3%	81.5%	76.3%
			CPU $\times$	0.34	0.99	1.30	1.10	0.93
			GPU $\times$	0.08	0.17	0.42	0.30	0.32
5	SSL	42.53%	column sparsity	0.00%	20.9%	39.7%	39.7%	24.6%
			CPU $\times$	1.00	1.27	1.64	1.68	1.32
			GPU $\times$	1.00	1.25	1.63	1.72	1.36

- Special libraries is required for sparsity network.
- It does not guarantee speedup.
- Structured sparsity is the way to go.
 - channel/layer/block pruning

<https://arxiv.org/pdf/1708.06519.pdf>

Channel pruning

- Steps
 - Select channel to prune
 - estimate the error
- Remove channel might not really improve the performance.
- It's hard to estimate the impact of the channel removal until we use dataset to evaluate it.
 - How to select the channel?
 - We may need to retrain the network after pruning the network

<https://github.com/Lam1360/YOLOv3-model-pruning>

Channel pruning: Generic method

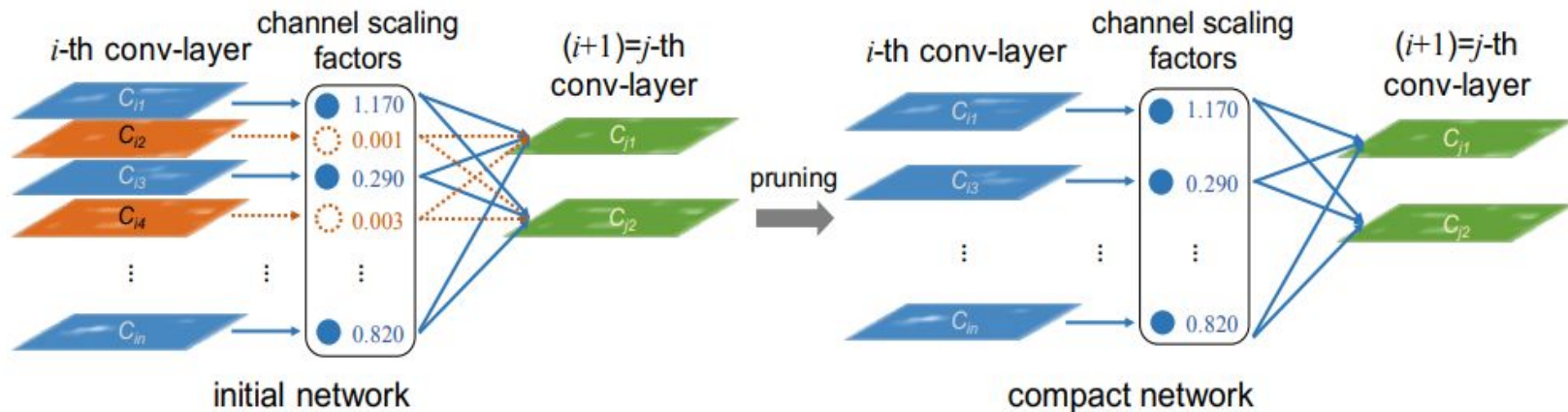


Figure 1: We associate a scaling factor (reused from a batch normalization layer) with each channel in convolutional layers. Sparsity regularization is imposed on these scaling factors during training to automatically identify unimportant channels. The channels with small scaling factor values (in orange color) will be pruned (left side). After pruning, we obtain compact models (right side), which are then fine-tuned to achieve comparable (or even higher) accuracy as normally trained full network.

Channel pruning: Use gamma in BN layer

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Use this factor as the channel scaling factor.

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Force the gamma become smaller

$$L = \sum_{(x,y)} l(f(x,W), y) + \lambda \sum_{\gamma \in \Gamma} g(\gamma)$$

put this term into the loss function to force the gamma of the BN layers become as small as possible.

Otherwise, small gamma and small filter are equivalent.

Prune the channels whose gamma is small

- After training with the modified loss function, we can simply remove the channels whose gamma is smaller than some threshold.
- Then, we can retrain the simplified model to improve the accuracy.

Test on CIFA-10

(a) Test Errors on CIFAR-10

Model	Test error (%)	Parameters	Pruned	FLOPs	Pruned
VGGNet (Baseline)	6.34	20.04M	-	7.97×10^8	-
VGGNet (70% Pruned)	6.20	2.30M	88.5%	3.91×10^8	51.0%
DenseNet-40 (Baseline)	6.11	1.02M	-	5.33×10^8	-
DenseNet-40 (40% Pruned)	5.19	0.66M	35.7%	3.81×10^8	28.4%
DenseNet-40 (70% Pruned)	5.65	0.35M	65.2%	2.40×10^8	55.0%
ResNet-164 (Baseline)	5.42	1.70M	-	4.99×10^8	-
ResNet-164 (40% Pruned)	5.08	1.44M	14.9%	3.81×10^8	23.7%
ResNet-164 (60% Pruned)	5.27	1.10M	35.2%	2.75×10^8	44.9%

Test on CIFAR-100

(b) Test Errors on CIFAR-100

Model	Test error (%)	Parameters	Pruned	FLOPs	Pruned
VGGNet (Baseline)	26.74	20.08M	-	7.97×10^8	-
VGGNet (50% Pruned)	26.52	5.00M	75.1%	5.01×10^8	37.1%
DenseNet-40 (Baseline)	25.36	1.06M	-	5.33×10^8	-
DenseNet-40 (40% Pruned)	25.28	0.66M	37.5%	3.71×10^8	30.3%
DenseNet-40 (60% Pruned)	25.72	0.46M	54.6%	2.81×10^8	47.1%
ResNet-164 (Baseline)	23.37	1.73M	-	5.00×10^8	-
ResNet-164 (40% Pruned)	22.87	1.46M	15.5%	3.33×10^8	33.3%
ResNet-164 (60% Pruned)	23.91	1.21M	29.7%	2.47×10^8	50.6%

Summary

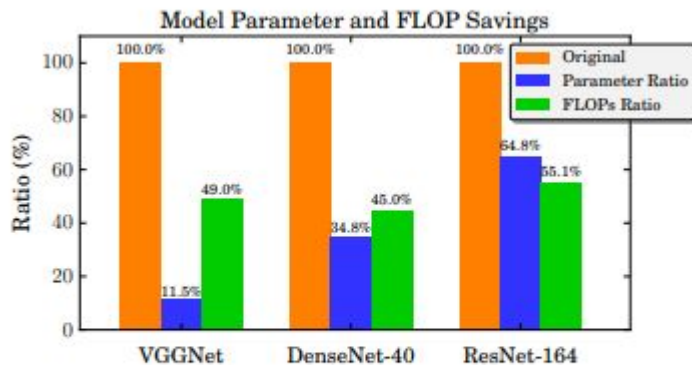


Figure 3: Comparison of pruned models with **lower** test errors on CIFAR-10 than the original models. The blue and green bars are parameter and FLOP ratios between pruned and original models.

Interactive pruning

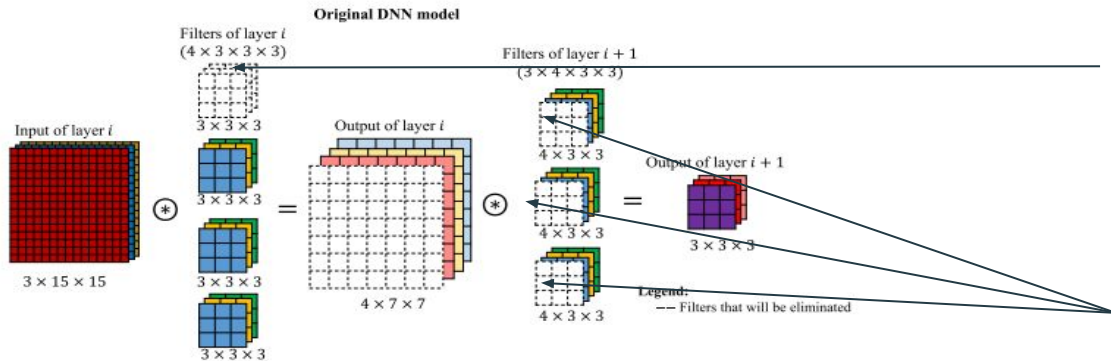
(a) Multi-pass Scheme on CIFAR-10

Iter	Trained	Fine-tuned	Params Pruned	FLOPs Pruned
1	6.38	6.51	66.7%	38.6%
2	6.23	6.11	84.7%	52.7%
3	5.87	6.10	91.4%	63.1%
4	6.19	6.59	95.6%	77.2%
5	5.96	7.73	98.3%	88.7%
6	7.79	9.70	99.4%	95.7%

(b) Multi-pass Scheme on CIFAR-100

Iter	Trained	Fine-tuned	Params Pruned	FLOPs Pruned
1	27.72	26.52	59.1%	30.9%
2	26.03	26.52	79.2%	46.1%
3	26.49	29.08	89.8%	67.3%
4	28.17	30.59	95.3%	83.0%
5	30.04	36.35	98.3%	93.5%
6	35.91	46.73	99.4%	97.7%

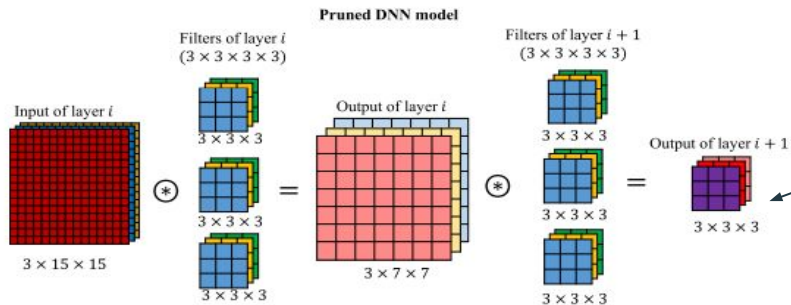
Filter Pruning:



We may choice to remove one set of $3 \times 3 \times 3$ filters from the network

However, since the shape of feature map at the next layer is changed, we need to remove those filters as well

Figure 3: Example of convolutional filter pruning where the first filter of the current convolution layer is selected to be eliminated.

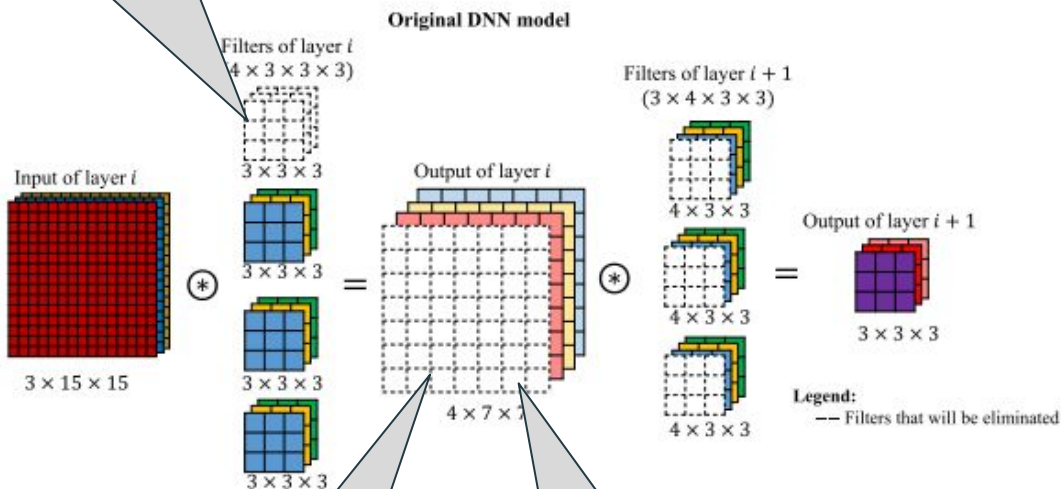


Therefore, we get this new network structure

Figure 4: Pruned DCNN model after the first filter is removed.

How do we choose the filters to be removed?

filter L1 norm



Mean activations

Average Percentage
of Zeros

Entropy: Use multiple image to provide a distribution. If the output of different image are the same, it means that the filters are not important.

Random

Evolutional algorithms

Bayesian algorithms

Layer pruning

- It's hard to remove the layer since it may spoil the network structure.
- We can apply it to the resnet
 - Keep the shortcut and remove the CNN blocks

<https://arxiv.org/pdf/2506.11469v1>

<https://www.mdpi.com/2504-4990/7/4/110>

dynamic pruning

- Use another network to select the pruning target
- Automatically select the better target by using network instead of using zero parameters.
- In principle, no fine tune after pruning.

Auto-Train-Once: Controller Network Guided Automatic Network Pruning from Scratch

- Use a mask to decide if we want to keep a channel or not.
- Each item in the mask will be mapped to a zero-invariant groups (ZIGs).
 - BN is a good target for CNN. $BN(X) = \gamma \cdot N(X) + \beta$
 - both parameters should be zeros.
- Instead of forcing paraterms to be zero directly, we use a gating network to decide the mask
 - If mask is 0, we will remove the channel from the network
- The gating network will be trained by using a subset of dataset.
 - The CNN and gating network are trained iteratively.
 - The loss will consider the FLOPS. We expect to remove the channels with bigger feature maps.

<https://arxiv.org/pdf/1803.03635.pdf>

<https://chatgpt.com/share/699af57a-9144-800a-8980-b232e7b3ecc0>

Why don't we use a smaller network?

- Typically, we can prune a big neural network to 5-10% of the size and maintain the same accuracy.
- It's easier to train a big network and then prune it.
- Lottery ticket hypothesis
 - A big network can be thought as an ensemble of smaller subnetworks.
 - When we train the network, each of them has some probability to be trained to have good performance.
 - If any of them win, we can get good performance.
 - Therefore, a big network has higher probability to win the prize.
- Therefore, we should train a big network and prune it.

RETHINKING THE VALUE OF NETWORK PRUNING

Zhuang Liu^{1*}, Mingjie Sun^{2*†}, Tinghui Zhou¹, Gao Huang², Trevor Darrell¹

¹University of California, Berkeley ²Tsinghua University

ABSTRACT

Network pruning is widely used for reducing the heavy inference cost of deep models in low-resource settings. A typical pruning algorithm is a three-stage pipeline, i.e., training (a large model), pruning and fine-tuning. During pruning, according to a certain criterion, redundant weights are pruned and important weights are kept to best preserve the accuracy. In this work, we make several surprising observations which contradict common beliefs. For all state-of-the-art structured pruning algorithms we examined, fine-tuning a pruned model only gives comparable or worse performance than training that model with randomly initialized weights. For pruning algorithms which assume a predefined target network architecture, one can get rid of the full pipeline and directly train the target network from scratch. Our observations are consistent for multiple network architectures, datasets, and tasks, which imply that: 1) training a large, over-parameterized model is often not necessary to obtain an efficient final model, 2) learned “important” weights of the large model are typically not useful for the small pruned model, 3) the pruned architecture itself, rather than a set of inherited “important” weights, is more crucial to the efficiency in the final model, which suggests that in some cases pruning can be useful as an architecture search paradigm. Our results suggest the need for more careful baseline evaluations in future research on structured pruning methods. We also compare with the “Lottery Ticket Hypothesis” (Frankle & Carbin, 2019), and find that with optimal learning rate, the “winning ticket” initialization as used in Frankle & Carbin (2019) does not bring improvement over random initialization.

Network architecture search vs pruning

Network pruning reduces the computation costs of an over-parameterized network without performance damage. Prevailing pruning algorithms pre-define the width and depth of the pruned networks, and then transfer parameters from the unpruned network to pruned networks. To break the structure limitation of the pruned networks, we propose to apply neural architecture search to search directly for a network with flexible channel and layer sizes. The number of the channels/layers is learned by minimizing the loss of the pruned networks. The feature map of the pruned network is an aggregation of K feature map fragments (generated by K networks of different sizes), which are sampled based on the probability distribution. The loss can be back-propagated not only to the network weights, but also to the parameterized distribution to explicitly tune the size of the channels/layers. Specifically, we apply channel-wise interpolation to keep the feature map with different channel sizes aligned in the aggregation procedure. The maximum probability for the size in each distribution serves as the width and depth of the pruned network, whose parameters are learned by knowledge transfer, e.g., knowledge distillation, from the original networks. Experiments on CIFAR-10, CIFAR-100 and ImageNet demonstrate the effectiveness of our new perspective of network pruning compared to traditional network pruning algorithms. Various searching and knowledge transfer approaches are conducted to show the effectiveness of the two components. Code is at: <https://github.com/D-X-Y/NAS-Projects>.

pruning by NAS

knowledge distillation vs
training from scratch or
finetune

<https://towardsdatascience.com/demystifying-the-lottery-ticket-hypothesis-in-deep-learning-158570b62674>

We don't know who is right yet

- We hope that we can find some knowledge which enable us to find the lottery.
- We hope to use a much smaller network for huge network, such as the GPT.

Bit precision

FP16 

INT8 

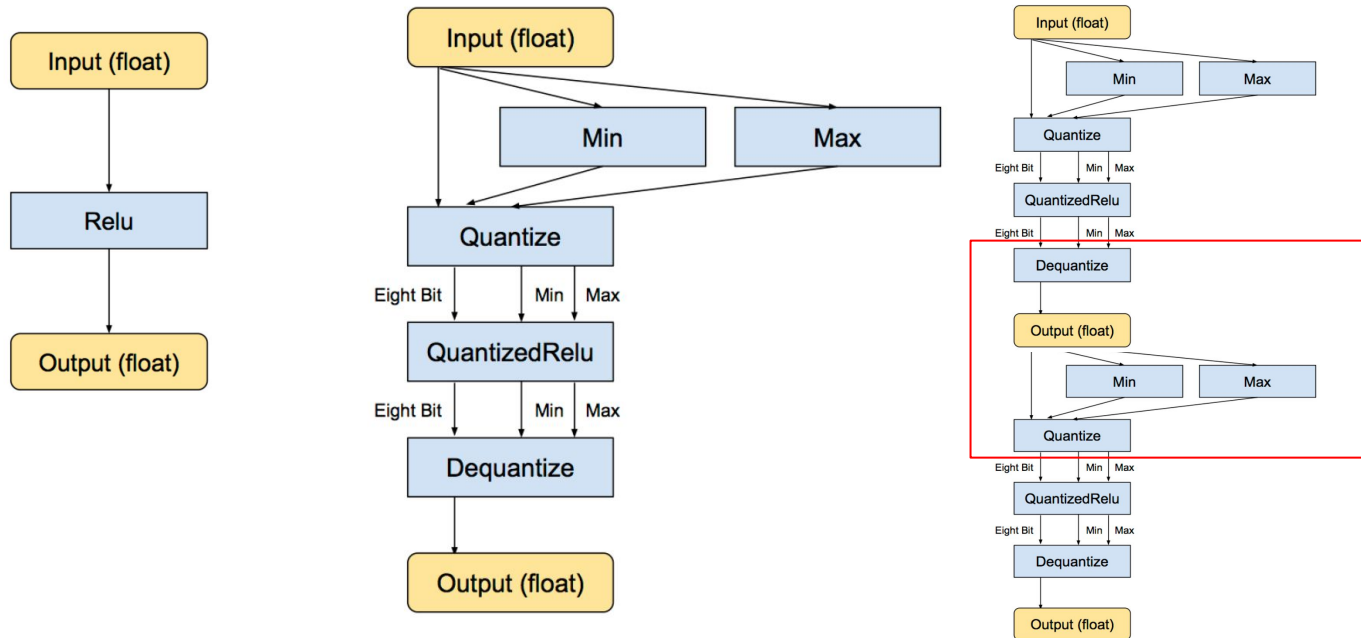
INT4 

INT2 

binarization 

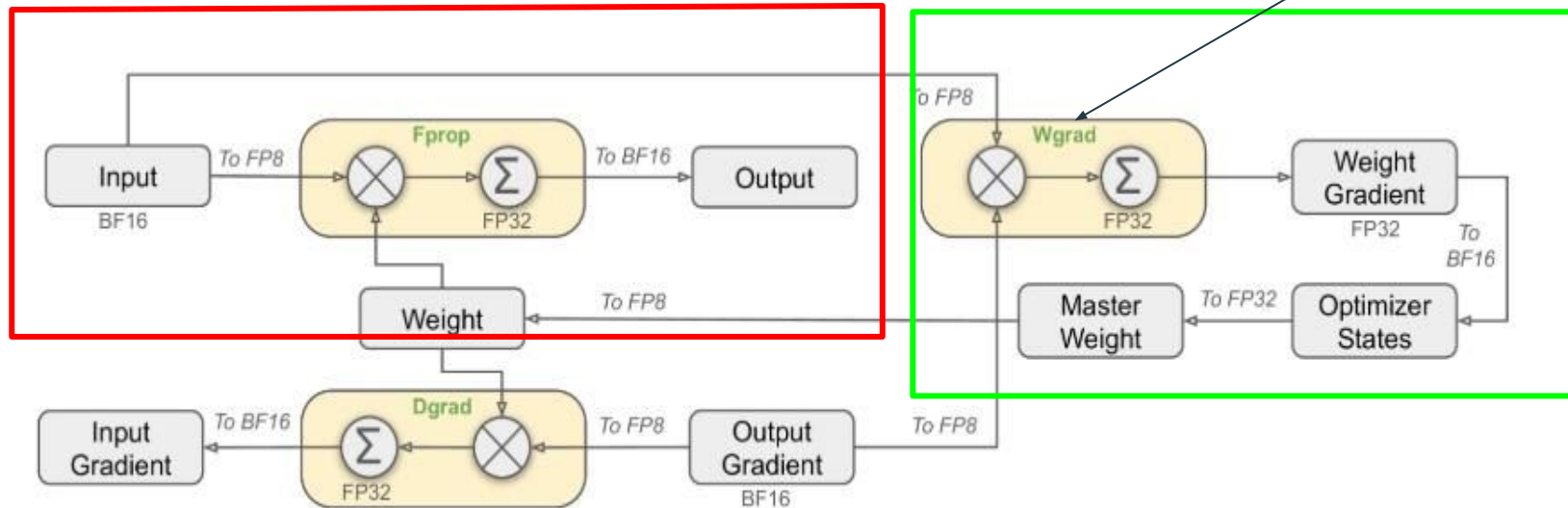
<https://www.youtube.com/watch?v=l2jyri5rgwY>

INT8 quantization



$$Y = X \cdot W$$

$$W = W - \eta X^T \frac{dL}{dY}$$



$$\frac{dL}{dX} W^T$$

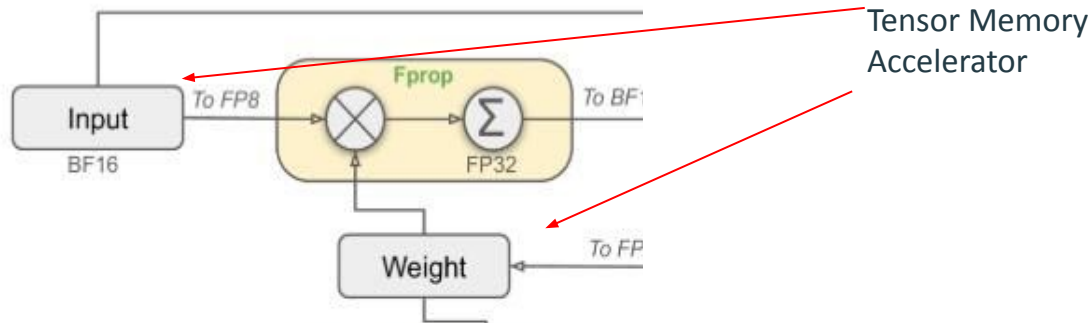
<https://github.com/deepseek-ai/DeepGEMM>

記憶體傳送和CUDA計算

$$Y = XW$$

$$dX = dYW^T$$

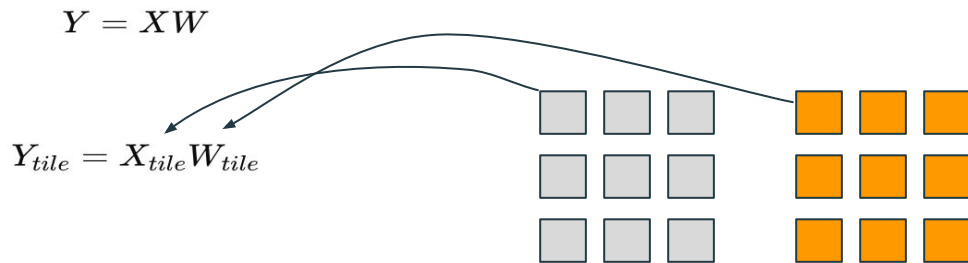
$$dW = X^T dY$$



都可以用同一個概念加速：把 **global memory** 到 **shared memory** 的資料搬移，和 **Tensor Core** 的**矩陣乘法**同時進行。在 NVIDIA Hopper/Blackwell 這類架構上，這件事主要靠 **TMA, Tensor Memory Accelerator** 完成。TMA 的目的就是有效率地把多維 tensor block 從 global memory 搬到 shared memory，而且這種 copy 可以和 kernel 內的計算重疊進行

以 forward 為例：

假設我們要算一個 output tile：

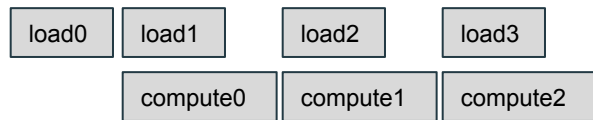


矩陣乘法通常會沿著 K 維度切成很多小塊：

$$Y_{ij} = \sum_k X_{ik} W_{kj}$$

所以 kernel 會反覆做：

1. 從 global memory 載入一塊 X
2. 從 global memory 載入一塊 W
3. 放到 shared memory
4. Tensor Core 做矩陣乘法
5. 再載入下一塊 X, W
6. 再計算下一塊



如果「搬資料」和「計算」是串接的，就會變成：

load $\rightarrow$ compute $\rightarrow$ load $\rightarrow$ compute

這樣 Tensor Core 在等資料時會空轉。

Better quantization- KL divergence

We want to find pairs of better (min.max) for each layer.

<https://arxiv.org/pdf/1902.06822.pdf>

How about INT4

We can get similar results as INT8.

<https://arxiv.org/pdf/1602.02830.pdf>

<https://www.mdpi.com/2079-9292/8/6/661/pdf>

<https://arxiv.org/abs/2310.11453>

Binarization Neural Network

- Use only 1-bit for the weights.
- It dramatically reduce memory and computation cost.
 - XNOR only
- The performance is downgraded.
- We need to finetune it to get better performance.

<https://youtu.be/XD5QO7xOYyl?list=PLj6E8qlqmkFv3cCjjX2SA1D4FJ9fadDij&t=201>

1-bit 怎麼算

先量化, 把 x 和 w 都變成 1-bit。

$$x \rightarrow x_{bit} \quad w \rightarrow w_{bit} \quad \sum x \cdot w \sim \alpha \sum x_{bit} \cdot w_{bit}$$

x_{bit}	w_{bit}	$x_{bit} \cdot w_{bit}$	XNOR	XOR
1(1)	1(1)	1(1)	1	0
1(1)	-1(0)	-1(0)	0	1
-1(0)	1(1)	-1(0)	0	1
-1(0)	-1(0)	1(1)	1	0

$$\begin{aligned} \sum x_{bit} w_{bit} &= \text{number of same bits} - \text{number of different bits} \\ &= XNOR - XOR \\ &= XNOR - (N - XNOR) \\ &= 2 \cdot XNOR - N \end{aligned}$$

BNN performance

Data set	MNIST	SVHN	CIFAR-10
Binarized activations+weights, during training and test			
BNN (Torch7)	1.40%	2.53%	10.15%
BNN (Theano)	0.96%	2.80%	11.40%
Committee Machines' Array (Baldassi et al., 2015)	1.35%	-	-
Binarized weights, during training and test			
BinaryConnect (Courbariaux et al., 2015)	1.29± 0.08%	2.30%	9.90%
Binarized activations+weights, during test			
EBP (Cheng et al., 2015)	2.2± 0.1%	-	-
Bitwise DNNs (Kim & Smaragdis, 2016)	1.33%	-	-
Ternary weights, binary activations, during test			
(Hwang & Sung, 2014)	1.45%	-	-
No binarization (standard results)			
Maxout Networks (Goodfellow et al.)	0.94%	2.47%	11.68%
Network in Network (Lin et al.)	-	2.35%	10.41%
Gated pooling (Lee et al., 2015)	-	1.69%	7.62%

Sparse Representation

- Quantization
- Multiplexing
- Weight Sharing

Sparse Representation: Quantization

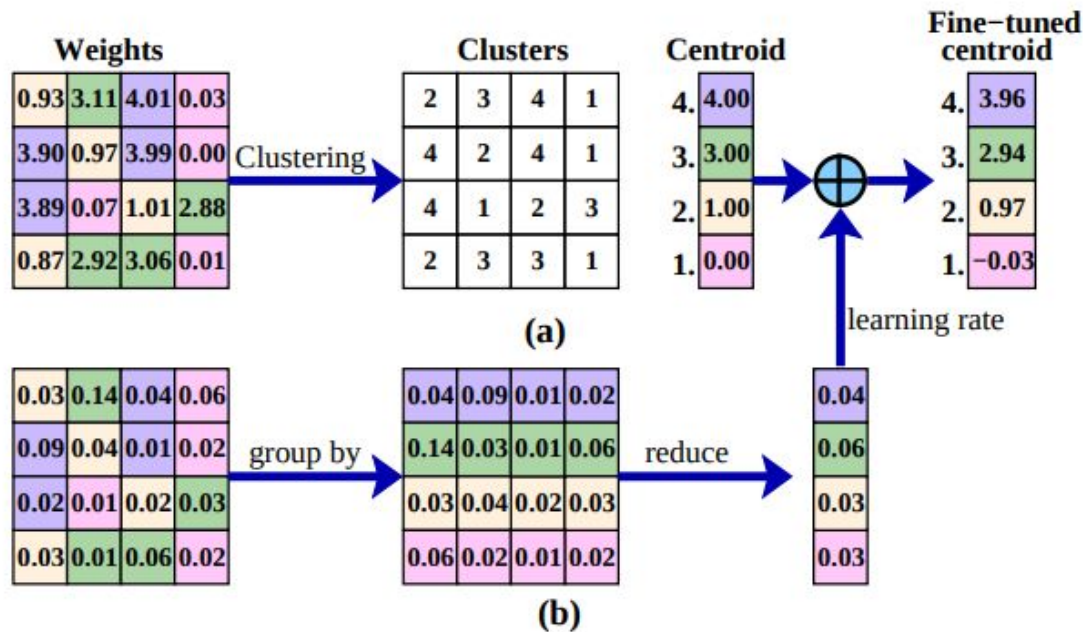
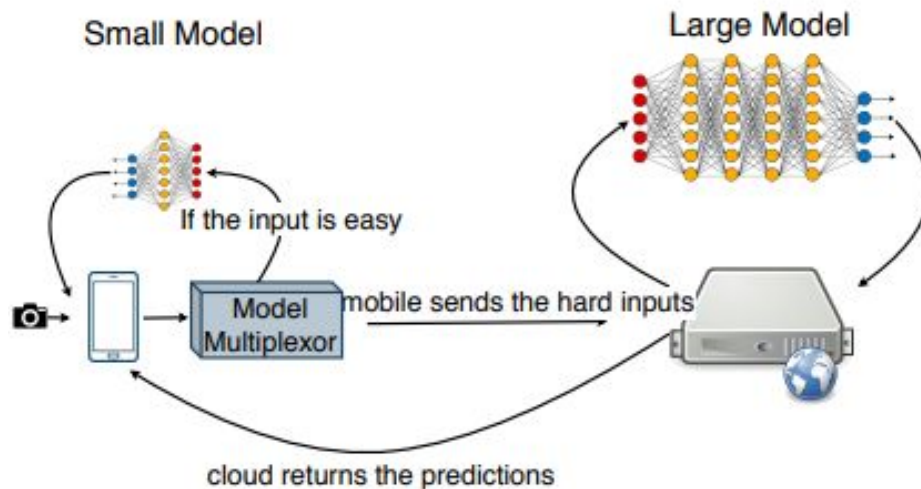


Fig. 5: Illustration of weight quantization using clustering [35]. Part (a) Forward propagation and part (b) backward propagation.

Sparse Representation: Multiplexing

- Use multiple models with different complexity
- Use a multiplexer to select the model for inference based on the input



Sparse Representation: Weight Sharing

- Use lower bits to represent the weights
- Use the same weights for different models
 - audio recognition
 - speaker identification
 - emotion recognition

Knowledge Distillation

- Use a smaller network to mimic the bigger network
- We can combine it with other methods to produce a tiny network

